Object Oriented Programming

Chapter 3:

Encapsulation and Overloading

Objectives

At the end of this Lecture, students will become familiar with:

- **Using access specifiers to encapsulate class members;**
- **Overloading the class constructor;**
- **Operator overloading.**

Outline

- Introduction
- Encapsulation
- Constructor overloading
- Operator overloading

Introduction

- Procedural programming is based on the availability of many procedures that call each other and share a lot of data, any one of them can modify these data.
- The procedural programming is suitable in the case of relatively small applications in terms of the number of lines of source code, and becomes impractical for medium and large applications. In such applications, making any change on the shared data requires a lot of caution, and requires thinking about all the procedures that share the data.
- In object-oriented programming, data belonging to an object and the methods using these data are grouped into the same object (within a single structure). It became possible to control access to the data of each object through the **encapsulation** principle.
- Object-oriented programming supports strongly the code reusability. It is normal for an object in one class to use members of another object, and it is normal to have several forms of the class constructor. This is called **Constructor Overloading**.

Encapsulation

- Encapsulation means preventing a class members from accessing the members of another class. Access to the members of each class can be controlled by using the appropriate **Access Specifier** for each of them.

Description	Access Specifier
Access is limited to members within the class	**Private**
Access is limited to members within the class and to members of classes inherited from that class	**Protected**
Access is limited to components within the same assembly that includes the class	**Internal**
Access is unlimited for all members of classes that have access to the class	**Public**

- When the access specifier of a class members is not declared, the access specifier is **private** by default;
- For the whole class, when the access specifier is not declared, it is **internal** by default, i.e. it can be accessed from within its assembly. An **assembly** is a piece of software (with the extension .exe or .dll) that is produced by compiling the source code and is written in the Microsoft intermediate language MSIL.

Encapsulation

```
1.  using System;
2.   namespace Encapsulation {
3.       class Rectangle   {
4.           //member variables
5.           double length;
6.           double width;
7.           // membwe method
8.           double GetArea()   {
9.               return length * width;
10.          }
11.      }//end class Rectangle

12.  class RectangleTest  {
13.      static void Main()    {
14.   Rectangle rec = new Rectangle();
15.       // accessing rec members
16.               rec.length = 5.0;
17.               rec.width = 4.0;
18.               Console.WriteLine("Area: {0}", rec.GetArea());
19.               Console.ReadLine();
20.           }// end Main
21.       }// end class RectangleTest
22.  }// end namespace Encapsulation
```

- And when we try to execute the code, we'll get the error messages:

'Rectangle.length' is inaccessible due to its protection level
'Rectangle.width' is inaccessible due to its protection level
'Rectangle.GetArea()' is inaccessible due to its protection level

Encapsulation

- The preceding messages means that the length, width, and GetArea elements of the Rectangle class cannot be accessed because the access specifiers of these members are private and cannot be accessed from outside the class.
- When we make access specifier to these members public, and re-implement the previous code, this issue is resolved, and the members of the rectangle become accessible, so we get the following result:

```
Area: 20
```

Constructor Overloading

- The constructor of a class is called when an object is created from it, and the constructor executes instructions for initializing the object.
- The class may contain several constructors, all of them have the same name and differ in the number of input arguments and/or the types of their input arguments;
- So, each constructor has its own way to configure the object, and this case is called **Constructor Overloading**.

```
using System;
namespace ConstructorOverloading {
    class Time  {
   private int hour; // 0 - 23
   private int minute; // 0 - 59
   private int second; // 0 - 59
 // constructor with zero argument
   public Time()   {
     // invoke SetTime to validate time 00:00:00
SetTime(0, 0, 0);
        } // end Time default constructor
  //constructor with one argument
   public Time(int h) {
   SetTime(h, 0, 0); // invoke SetTime to validate time
        } // end constructor with one argument
```

```
//constructor with two arguments
 public Time(int h, int m) {
  // invoke SetTime to validate time
SetTime(h, m, 0);
 } // end constructor with two arguments
 //constructor with three arguments

 public Time(int h, int m, int s) {
  SetTime(h, m, s);
        } // end constructor with three
arguments
```

Constructor Overloading

```
// Time constructor: another Time object supplied as an argument
        public Time(Time time) : this(time.Hour,
                                      time.Minute, time.Second) { }
        // set a new time value using universal time; ensure that
        // the data remains consistent by setting invalid
        // values to zero
       public void SetTime(int h, int m, int s)
        {
            Hour = h; // set the Hour property
            Minute = m; // set the Minute property
            Second = s; // set the Second property
      } // end method SetTime
// property that gets and sets the hour
        public int Hour {
            get {
                return hour;
            } // end get
            set {
                if (value >= 0 && value < 24)
                    hour = value;
                else
         Console.WriteLine("Hour  " + value + "  Hour must be 0-23");
            } // end set
        } // end property Hour
```

Constructor Overloading

```
1.  // property that gets and sets the minute
2.          public int Minute {
3.              get { return minute; } // end get
4.              set
5.              {
6.                  if (value >= 0 && value < 60)
7.                      minute = value;
8.                  else
9.           Console.WriteLine("Minute  " + value + "  Minute must be 0-59");
10.             } // end set
11.         } // end property Minute
12.   // property that gets and sets the second
13.         public int Second {
14.             Get  {
15.                 return second;
16.             } // end get
17.             Set {
18.                 if (value >= 0 && value < 60) second = value;
19.                 else
20.          Console.WriteLine("Second  " + value + "  Second must be 0-59");
21.             } // end set
22.         } // end property Second
```

Constructor Overloading

```
1.  // convert to string in universal-time format (HH:MM:SS)
2.          public string ToUniversalString() {
3.      return string.Format("{0:D2}:{1:D2}:{2:D2}", Hour, Minute, Second);
4.          } // end method ToUniversalString
5.
6.          // convert to string in standard-time format (H:MM:SS AM or PM)
7.          public string ToStandardString() {
8.              return string.Format("{0}:{1:D2}:{2:D2} {3}",
9.                  ((Hour == 0 || Hour == 12) ? 12 : Hour % 12),
10.                  Minute, Second, (Hour < 12 ? "AM" : "PM"));
11.         } // end method ToStandardString
12.
    } // end class Time
```

Constructor Overloading

```
1.  public class TimeTest {
2.          public static void Main() {
3.              Time t1 = new Time(); // 00:00:00
4.              Time t2 = new Time(2); // 02:00:00
5.              Time t3 = new Time(21, 34); // 21:34:00
6.              Time t4 = new Time(12, 25, 42); // 12:25:42
7.              Time t5 = new Time(t4); // 12:25:42
8.              Console.WriteLine("t1: all arguments defaulted");
9.              Console.WriteLine(" {0}", t1.ToUniversalString());        // 00:00:00
10.             Console.WriteLine(" {0}\n", t1.ToStandardString());      // 12:00:00 AM
                Time t6;  // initialized later in the program
11.                t6 = new Time(27, 74, 99); // invalid values become 00:00:00
12.             Console.WriteLine(" {0}", t6.ToUniversalString())
13. } // end Main
14.     } // end class TimeTest
15. }// namespace ConstructorOverloading
```

Constructor Overloading

After executing the previous code, we get the following output:

Note: When one or more constructors are defined by the programmer, the default constructor (which has no input arguments) must be defined if we need it.

```
Constructed with:

t1: all arguments defaulted
 00:00:00
 12:00:00 AM

t2: hour specified; minute and second defaulted
 02:00:00
 2:00:00 AM

t3: hour and minute specified; second defaulted
 21:34:00
 9:34:00 PM

t4: hour, minute and second specified
 12:25:42
 12:25:42 PM

t5: Time object t4 specified
 12:25:42
 12:25:42 PM
Hour  27  Hour must be 0-23
Minute  74  Minute must be 0-59
Second  99  Second must be 0-59
 00:00:00
```

Operator Overloading

- Some operators can be reused to perform functions specified by the programmer.
- These operators gain the ability to perform additional functionality, which is called **Operator Overloading.**
- **Example:** It is known that the addition operator (+) is used to add numerical values, and it can also be used to concatenate strings together.
- When any of these operators is called, the corresponding method is called. The methods corresponding to the operators are public static, and their names begin with the word "**operator**", so the method corresponding to the (+) operator, for example, is **operator+**.

Operator Overloading

```
1.  public class ComplexNumber  {
2.          // read-only property that gets the real component
3.          public double Real { get; private set; }
4.          // read-only property that gets the imaginary component
5.          public double Imaginary { get; private set; }
6.          // constructor
7.          public ComplexNumber(double r, double i) {
8.              Real = r; Imaginary = i;
9.          } // end constructor
10. // overloading the addition operator
11.  public static ComplexNumber operator+ (ComplexNumber x, ComplexNumber y) {
12.
13.    return new ComplexNumber(x.Real + y.Real, x.Imaginary + y.Imaginary);
14.
15.         } // end operator +
```

Operator Overloading

```
1. public static void Main( ) {
2.                 // declare two variables to store complex numbers
3.                 // to be entered by user
4.                 ComplexNumber x, y,z;
5.                 x = new ComplexNumber(5, 3);
6.                 y = new ComplexNumber(8, -2);
7.                 Z = x + y;
8. } // end method Main
```

Summary

- **Encapsulation**
- **Constructor overloading**
- **Operator overloading**

Thank you very much for attending

You can kindly ask questions